Short Answer Question [2 Marks]

1. Explain DOM vs HTML.
2. Define the DOM and explain its role in web development.
3. What is DOM? Explain DOM tree structure.
4. Explain getElementById() and getElementsByClassName().
5. What is callback function? Explain its drawbacks.
6. Explain the concept of event propagation in JavaScript.
7. Differentiate between event bubbling and event capturing.
8. Explain Temporal Dead Zone (TDZ) in JavaScript with a suitable example.
9. Compare localStorage and sessionStorage in JavaScript. Give one use case for each.
10. Give the explanation of classList methods (add/remove/toggle).
11. What are promise states in JavaScript?
12. Discuss microtask and macrotask queue.
13. Examine the role of Babel in converting modern JavaScript code.
14. Define props in React with a simple use case.
15. What is JSX in React?
16. What is one-way data binding?
17. Explain the concept of Virtual DOM and how it differs from the Real DOM in React.
18. Define props in React. How are props used to pass data between components?
19. Explain the reconciliation process in React and its role in improving performance.
20. What are React Hooks? Why were they introduced in React?
21. Demonstrate the role of useState using a simple example.
22. Explain Virtual DOM vs Real DOM and SPA.
23. What is useEffect? Explain side effects.
24. How React manage frequent changes done by user effectively?
25. Elaborate the use of dependancy array ?
26. What is Context API?

Long Answer Question [8 Marks]

Q.1 Write a complete program using HTML and JavaScript to perform the following tasks:

- Create an HTML element with some initial text.
- Select the element
- Change its background color to blue.
- Change its text to "Hello DOM"

Q.2 Write a program using HTML and JavaScript to:

- Create a form with three input fields: name, age, gender and a submit button
- Handle the form submit event
- Use preventDefault() to stop page reload
- Store the input values in an object
- Display the object in the console

Q.3 What are DOM selectors in JavaScript? Explain different types of selectors used to select HTML Elements with suitable examples. Also, differentiate between getElementsByClassName() and querySelectorAll().

Q.4 Create a webpage with a button and a paragraph. Write JavaScript code using addEventListener() to:

- Change the text of the paragraph to "Button Clicked!" when the button is clicked
- Change the text color of the paragraph to blue on click
- Change the text color to red when the mouse is over the paragraph

Q.5 Explain event propagation. Differentiate between bubbling and capturing. Discuss the role of event listeners in handling user interactions on webpages.

Q.6 Create a simple webpage using HTML, CSS, and JavaScript that demonstrates button click events and event removal.

- Add a button on the webpage.
- Add a text paragraph/heading with some default text.
- When the button is clicked:
  - The text should change to a new message.
  - The text color should also change.

After performing the action once, use removeEventListener() so that the button click event is removed.

Q.7 Discuss event handling in JavaScript using addEventListener() with different event types.

Q.8 Explain execution context in JavaScript including GEC, FEC, and call stack with diagram.

Q.9 Explain Promises in JavaScript. Discuss states and promise consumers (.then(), .catch(), .finally()). Explain with example

Q.10 Write a JavaScript program using setTimeout() and setInterval():

- Execute a task after delay
- Repeat execution
- Stop it using clearTimeout()

Q.11 What is a callback function? Explain nested callbacks and their drawbacks with examples.

Q. 12 Explain Promises. Create a Promise that:

- Resolves after 2 seconds
- Prints a success message using .then()
- Handles errors using .catch()
- Uses .finally()

Q. 13 Write the code-- how to fetch API In React (https://jsonplaceholder.typicode.com/users)

Sample JSON:

```
{
  "username": "Bret",
  "email":"Sincere@april.biz",
}
```

- Fetch data for all users

- Render username and email of each user
- Display results (Output)

Q. 14 Explain localStorage, sessionStorage, and cookies with proper examples.

Q. 15 Demonstrate async/await with error handling using try...catch.

Q. 16 a) Write the output.
b) Explain the execution flow with respect to synchronous and asynchronous behavior.

```
console.log("Start");

async function test() {
console.log("Inside function");

await Promise.resolve();

console.log("After await");
}

test();

setTimeout(() => {
console.log("Timeout");
}, 0);

console.log("End");
```

Q. 17 Write a React program to create a simple Counter Application:

- Use the useState hook to store a count value
- Display the current count on the screen
- Add two buttons:
  - Increment → increases the count by 1
  - Decrement → decreases the count by 1
- Ensure that the count updates correctly on each button click

Q. 18 Create a React component:

- Use useState
- Pass data as props
- Use conditional rendering (ternary)

Q. 19 Explain one-way data binding and component-based architecture in React. Also explain default and named exports.

Q. 20 Explain state and props in React. Create a React component that:

- Uses useState to manage a counter
- Passes data as props to a child component

Q. 21 Explain component lifecycle methods in React and how we implement lifecycle methods in UseEffect hook with examples.

Q. 22 Create a React component with two buttons and a counter state.

- When the first button is clicked:
- Increase the counter by 1
- Show a popup/alert every time using useEffect
- When the second button is clicked:
- Increase the counter by 1
- The counter should update, but no popup should appear

Q.23 Design and implement a User Information Form using React with the following requirements:

1) Create a form that includes the following input fields:
   Name (Text Input)
   Age (Number Input)
   Gender (Select Box)
   Language (Checkbox)

2) On form submission, display the entered data on the screen.

Q.24 Write a React program to:

- Fetch data from the API: https://jsonplaceholder.typicode.com/users
- Display "Loading..." while data is being fetched
- Store the fetched data using useState
- Display the received data (e.g., user names) in a list format

Q. 25 Explain React Router:

- Routes
- Link / NavLink
- useNavigate

Q. 26 Elaborate prop drilling and its problems. How does Context API solve it?

Q. 27 Explain form handling in React including input handling, submission, and synthetic events.
Write a React form that:

- Handles input fields
- Prevents default submission
- Displays entered data

Q. 28 Analyze the role of performance optimization techniques such as debouncing and pagination in React applications. Compare how each technique improves efficiency and user experience, and illustrate their working with suitable examples.

Q. 29 Compare the React Context API and its advantages over props drilling. Then, write a React program to create a context, provide data at the parent level, and access the same data in other hierarchical child components.

- Key Focus: Explain the concept of Context API and compare it with props drilling.
- Practical Requirement: Create, provide, and consume context data across nested components.